

Factory Traffic Management System

Backend Developer Intern Assessment — Complete Implementation Guide

Purpose: A practical, assessment-ready blueprint covering backend architecture, database schema, REST APIs, traffic state machine, scheduler, emergency/manual control, failure handling, testing, frontend dashboard, and README structure.

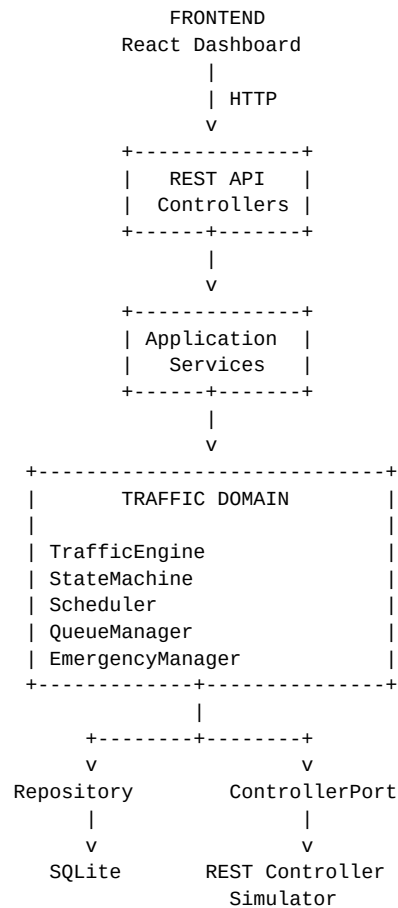
Assessment basis: This guide is derived from the uploaded Factory Traffic Management System — Backend Developer Intern Assessment V2. The assessment emphasizes clear architecture, safe traffic transitions, event-driven behavior, persistence/recovery, concurrency awareness, testable domain logic, and the candidate's ability to explain design decisions.

Source: Uploaded assessment, especially the requirements for separation of API/domain/persistence/controller/frontend layers and the warning that this is an event-driven control system rather than a simple CRUD application.

1. Recommended Technology Stack

Layer	Technology	Reason
Backend	Node.js + TypeScript + Express	Clear structure and strong typing
Validation	Zod	Runtime request validation
Database	SQLite	Fast setup and sufficient for the assessment
DB access	better-sqlite3	Simple synchronous SQLite access
Tests	Vitest	Fast unit/integration testing
Frontend	React + TypeScript + Vite	Simple dashboard implementation
IoT	REST controller simulator	Meets the assessment without physical hardware
Live updates	Polling	Simple and reliable; WebSocket/SSE is optional

2. Overall Architecture



The central design rule is that the traffic-control domain must not depend directly on HTTP, MQTT, frontend code, or a specific database. A controller port/interface allows a REST simulator today and MQTT later without rewriting the domain engine.

3. Project Structure

```
factory-traffic-management/  
  ■■■ backend/  
  ■   ■■■ src/  
  ■   ■   ■■■ config/  
  ■   ■   ■■■ domain/  
  ■   ■   ■   ■■■ enums/  
  ■   ■   ■   ■■■ entities/  
  ■   ■   ■   ■■■ TrafficEngine.ts  
  ■   ■   ■   ■■■ TrafficStateMachine.ts  
  ■   ■   ■   ■■■ TrafficScheduler.ts  
  ■   ■   ■   ■■■ QueueManager.ts  
  ■   ■   ■■■ application/  
  ■   ■   ■   ■■■ services/  
  ■   ■   ■   ■■■ dto/  
  ■   ■   ■■■ infrastructure/  
  ■   ■   ■   ■■■ database/  
  ■   ■   ■   ■■■ repositories/  
  ■   ■   ■   ■■■ controller/  
  ■   ■   ■■■ api/  
  ■   ■   ■   ■■■ routes/  
  ■   ■   ■   ■■■ controllers/  
  ■   ■   ■   ■■■ validators/  
  ■   ■   ■   ■■■ errorHandler.ts  
  ■   ■   ■■■ scheduler/  
  ■   ■   ■■■ app.ts  
  ■   ■   ■■■ server.ts  
  ■   ■■■ tests/  
  ■   ■■■ package.json  
  ■   ■■■ tsconfig.json  
  ■■■ frontend/  
  ■   ■■■ src/  
  ■■■ README.md  
  ■■■ .gitignore
```

4. Domain Model

Directions and signals

Directions:

NORTH
SOUTH
EAST
WEST

Physical signal states:

RED
YELLOW
GREEN

Internal traffic phases:

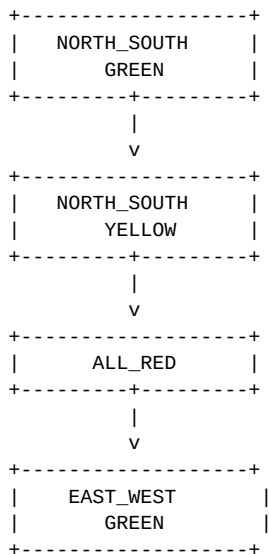
NORTH_SOUTH
EAST_WEST
ALL_RED

Operating modes:

AUTOMATIC
MANUAL
EMERGENCY
DEGRADED

The assessment permits an internal ALL_RED clearance state and requires conflicting phases never to receive GREEN simultaneously.

5. Traffic State Machine



Current	Request	Next
N/S GREEN	N/S	Remain N/S
N/S GREEN	E/W	N/S YELLOW
N/S YELLOW	E/W	ALL_RED
ALL_RED	E/W	E/W GREEN
E/W GREEN	N/S	E/W YELLOW
E/W YELLOW	N/S	ALL_RED
ALL_RED	N/S	N/S GREEN

Safety invariant: Never perform a direct transition from a GREEN phase to a conflicting GREEN phase. Manual and emergency requests must use the same safe transition mechanism.

6. Scheduler

A simple explainable scheduling score is recommended because the assessment leaves the exact weighting unspecified.

```

Vehicle priority:
EMERGENCY      = 100
TRUCK          = 70
FORKLIFT       = 40
EMPLOYEE_VEHICLE = 20
  
```

```

Direction score:
queueSize * 10
+ highestVehiclePriority
+ waitingTimeBonus
+ starvationBonus
  
```

One possible waiting-time rule is: $\text{waitingTimeBonus} = \min(\text{waitingSeconds} / 2, 50)$. This prevents a small queue from waiting forever while still giving queue size and vehicle priority meaningful influence. The exact weights should be documented as implementation decisions.

7. Emergency Handling

```
Current:
N/S GREEN
E/W RED
```

Emergency arrives from EAST

```
N/S GREEN
  |
  v
N/S YELLOW
  |
  v
ALL RED
  |
  v
E/W GREEN
```

Emergency traffic receives the highest priority but does not bypass the safety transition. For simultaneous emergencies on conflicting phases, use a deterministic policy such as earliest valid emergency first, then re-evaluate the remaining emergency. Emergencies on the same phase can be served together.

8. Manual Control

```
POST /api/junctions/A/commands
```

```
{
  "command": "MANUAL_GREEN_REQUEST",
  "direction": "WEST"
}
```

The API must request an intent, not directly write arbitrary signal states. The domain engine calculates the safe transition. RETURN_TO_AUTOMATIC explicitly ends the manual mode.

9. Database Schema

junctions

```
CREATE TABLE junctions (
  id TEXT PRIMARY KEY,
  name TEXT NOT NULL,
  mode TEXT NOT NULL DEFAULT 'AUTOMATIC',
  phase TEXT NOT NULL DEFAULT 'NORTH_SOUTH',
  controller_status TEXT NOT NULL DEFAULT 'ONLINE',
  created_at TEXT NOT NULL,
  updated_at TEXT NOT NULL
);
```

vehicles

```
CREATE TABLE vehicles (
  id TEXT PRIMARY KEY,
  junction_id TEXT NOT NULL,
  direction TEXT NOT NULL,
  vehicle_type TEXT NOT NULL,
  arrived_at TEXT NOT NULL,
  cleared_at TEXT,
  status TEXT NOT NULL DEFAULT 'WAITING',
  FOREIGN KEY (junction_id) REFERENCES junctions(id)
);
```

sensor_events

```
CREATE TABLE sensor_events (  
    event_id TEXT PRIMARY KEY,  
    junction_id TEXT NOT NULL,  
    vehicle_id TEXT NOT NULL,  
    event_type TEXT NOT NULL,  
    direction TEXT NOT NULL,  
    vehicle_type TEXT,  
    sequence_no INTEGER,  
    sensor_timestamp TEXT,  
    received_at TEXT NOT NULL,  
    status TEXT NOT NULL,  
    FOREIGN KEY (junction_id) REFERENCES junctions(id)  
);
```

controller_commands

```
CREATE TABLE controller_commands (  
    command_id TEXT PRIMARY KEY,  
    junction_id TEXT NOT NULL,  
    direction TEXT,  
    requested_state TEXT NOT NULL,  
    status TEXT NOT NULL,  
    created_at TEXT NOT NULL,  
    acknowledged_at TEXT,  
    timeout_at TEXT  
);
```

signal_states

```
CREATE TABLE signal_states (  
    junction_id TEXT NOT NULL,  
    direction TEXT NOT NULL,  
    desired_state TEXT NOT NULL,  
    actual_state TEXT NOT NULL,  
    updated_at TEXT NOT NULL,  
    PRIMARY KEY (junction_id, direction),  
    FOREIGN KEY (junction_id) REFERENCES junctions(id)  
);
```

audit_history

```
CREATE TABLE audit_history (  
    id INTEGER PRIMARY KEY AUTOINCREMENT,  
    junction_id TEXT NOT NULL,  
    event_type TEXT NOT NULL,  
    direction TEXT,  
    previous_state TEXT,  
    new_state TEXT,  
    command_id TEXT,  
    details TEXT,  
    timestamp TEXT NOT NULL,  
    FOREIGN KEY (junction_id) REFERENCES junctions(id)  
);
```

Desired and actual physical signal state must be separate. Sending a controller command does not automatically prove that the physical device executed it.

10. REST API

Method	Endpoint	Purpose
GET	/api/junctions	List junctions
GET	/api/junctions/:id	Get junction
POST	/api/junctions	Create junction
GET	/api/junctions/:id/status	Current status
POST	/api/sensor-events	Vehicle event
POST	/api/junctions/:id/commands	Manual/control command
POST	/api/controller-events	Controller ACK/failure
GET	/api/junctions/:id/history	Audit history

Example vehicle-arrival request

POST /api/sensor-events

```
{
  "event_id": "evt-10001",
  "junction_id": "A",
  "direction": "NORTH",
  "event_type": "VEHICLE_ARRIVED",
  "vehicle_id": "VH-501",
  "vehicle_type": "TRUCK",
  "sequence_no": 1501,
  "timestamp": "2026-10-05T10:15:20Z"
}
```

11. Idempotent Event Processing

```
Validate event
|
v
Check event_id
|
+---- already exists ----> DUPLICATE_EVENT
|
v
Store event
|
v
Update vehicle/queue
|
v
Write audit record
|
v
Trigger scheduler
```

The event_id is the primary idempotency key. A repeated event must not increase or decrease the queue a second time. Invalid, unknown, delayed, or out-of-order events should be rejected or handled according to the documented policy.

12. Controller Communication

Backend command:

```
{
  "command_id": "cmd-8001",
  "junction_id": "A",
  "direction": "EAST",
  "requested_state": "GREEN"
}
```

Controller acknowledgement:

```
{
  "command_id": "cmd-8001",
  "junction_id": "A",
  "status": "ACK",
  "actual_state": "GREEN"
}
```

Possible command statuses: PENDING, ACKNOWLEDGED, FAILED, TIMEOUT. Duplicate ACKs should be idempotent. If an ACK is never received, actual state remains unconfirmed and the junction can enter DEGRADED/FAILURE behavior.

13. Concurrency Strategy

```
Junction A
  |
  v
[Serialized event processing for A]
  |
  v
TrafficEngine

Junction B
  |
  v
[Serialized event processing for B]
  |
  v
TrafficEngine
```

Recommended approach: serialize state-changing operations per junction. This prevents a truck arrival, emergency request, manual request, and controller ACK from mutating the same junction state concurrently. Alternative valid approaches include database locking or optimistic concurrency.

14. Scheduler Worker

Do not use long blocking sleep() calls inside HTTP handlers. Instead, persist transition state and use a background worker/timer to evaluate junctions periodically.

```
Event/request
  |
  v
Persist state
  |
  v
Scheduler evaluates junction
  |
  v
State machine determines next transition
  |
  v
Controller command
  |
  v
Wait for ACK / deadline
  |
  v
Update actual state when confirmed
```

15. Frontend Dashboard

The frontend should display backend state rather than independently calculating traffic sequencing.

```
+-----+
| FACTORY TRAFFIC MANAGEMENT |
+-----+
| Junction A                  |
| Mode: AUTOMATIC             |
| Phase: NORTH / SOUTH        |
| Controller: ONLINE           |
|                               |
|         GREEN NORTH          |
|         |                    |
| GREEN SOUTH + RED EAST      |
|         |                    |
|         RED WEST            |
|                               |
| Queues                      |
| NORTH 7   SOUTH 3   EAST 9   WEST 2 |
|                               |
| [Manual Control] [Return Automatic] |
|                               |
| Emergency: NONE             |
| Failure: NONE               |
|                               |
| Recent Activity              |
+-----+
```

Recommended dashboard features: signal states, queue sizes, controller status, current mode, current phase, emergency/failure alerts, desired vs actual state, pending command, recent activity, manual controls, and a simulator for vehicle/controller events.

16. Testing Strategy

Test Area	Important Tests
State machine	All valid transitions; no conflicting GREEN states
Safety	No GREEN-to-conflicting-GREEN; manual/emergency use safe path
Events	Arrival, clearance, invalid data, unknown junction
Idempotency	Duplicate event does not change queue twice
Scheduler	Queue size, vehicle priority, waiting time, starvation protection
Emergency	Preemption, repeated emergency, conflicting emergencies
Controller	ACK, timeout, failure, duplicate ACK
Recovery	Restart preserves state/history; physical state not assumed
Concurrency	Near-simultaneous events remain consistent

17. Demo Scenarios

- Normal traffic: several vehicles arrive from different directions and the scheduler selects a safe phase.
- Priority traffic: a TRUCK/FORKLIFT affects scheduling compared with employee traffic.
- Emergency preemption: an emergency arrives while a conflicting phase is GREEN and the backend safely transitions.
- Manual override: an administrator requests a direction and later returns to automatic mode.
- Duplicate event: submit the exact same event twice; queue state remains correct.
- Vehicle clearance: arrival followed by VEHICLE_CLEARED reduces the queue.
- Controller failure: requested state is not ACKed or controller goes OFFLINE; system enters safe/degraded handling.
- Restart: restart backend and verify persisted queue/history and safe physical-state recovery.

- Concurrent events: NORTH truck, EAST emergency, WEST manual request, duplicate emergency, and controller ACK occur close together.

18. Assumptions / Questions / Requirement Issues

Issue	Implementation Decision
Conflicting movements	N/S and E/W are the two conflicting phases.
Duplicate events	event_id is the idempotency key.
Queue source	Queue is derived from valid vehicle events.
Negative queue	Never allowed; unmatched clearance is rejected/audited.
Timestamps	Store sensor timestamp and server-received timestamp.
Ordering	Use sequence_no when available; document delayed/out-of-order handling.
Emergency vs manual	Emergency has higher priority.
Manual duration	Manual remains active until explicit return to automatic.
Emergency conflict	Deterministic earliest-valid emergency policy, then re-evaluate.
Controller ACK	Actual state changes only after confirmation.
ACK timeout	Pending command becomes TIMEOUT and contributes to degraded handling.
Restart	Persist logical state but do not assume physical signal state is correct.
Live dashboard	Use polling initially; SSE/WebSocket is optional.
IoT	REST simulator now; MQTT can be a later adapter.

19. README Structure

```
# Factory Traffic Management System

## 1. Overview
## 2. Features
## 3. Architecture
## 4. Technology Stack
## 5. Project Structure
## 6. Database Design
## 7. Traffic-Control Algorithm
## 8. Traffic State Machine
## 9. Scheduling Strategy
## 10. Emergency Handling
## 11. Manual Control
## 12. Controller Communication
## 13. Failure Handling
## 14. Concurrency Strategy
## 15. API Documentation
## 16. Running the Project
## 17. Testing
## 18. Demo Scenarios

## Assumptions / Questions / Requirement Issues

## 19. Limitations / Future Work
## 20. AI / Tool Usage
```

20. Development Order

Phase 1 — Backend foundation

TypeScript/Express, SQLite, project structure, schema, seed Junction A.

Phase 2 — Core domain

QueueManager, StateMachine, Scheduler, TrafficEngine.

Phase 3 — API

Junction, sensor-event, command, controller-event, and history APIs.

Phase 4 — Reliability

Idempotency, concurrency protection, controller timeout, degraded mode, restart recovery.

Phase 5 — Tests

State machine, scheduler, events, emergency, controller, concurrency, recovery.

Phase 6 — Frontend

Dashboard, intersection visualization, queues, manual control, emergency/failure display, simulator, polling.

Phase 7 — Documentation

README, API examples, architecture, assumptions, demo scenarios, AI/tool usage.

21. Final Evaluation Checklist

- Core traffic logic is independent of HTTP, frontend, database, and MQTT.
- Conflicting GREEN states can never occur.
- GREEN-to-conflicting-GREEN is never a direct transition.
- Manual and emergency control use the same safe transition path.
- Duplicate sensor events are idempotent.
- Queue state is based on valid events and cannot become negative.
- Desired and controller-confirmed actual signal state are separate.
- Controller ACK/timeout/failure behavior is explicit.
- State-changing operations for a junction are serialized or otherwise concurrency-safe.
- Important state and audit history survive restart.
- Domain logic is unit-testable without HTTP or hardware.
- Frontend displays backend state and does not independently sequence traffic.
- README documents assumptions, requirement issues, architecture, algorithm, testing, and AI/tool usage.

22. Assessment Strategy

The strongest submission is not the one with the most screens. It is the one where an evaluator can trace an event from input to domain decision to safe state transition to controller command, see how the result is persisted, and understand what happens when the controller fails or the server restarts.

Keep the implementation intentionally small, deterministic, and explainable. Optional MQTT, WebSocket/SSE, Docker, authentication, analytics, metrics, and event replay should only be added after the core safety and consistency behavior is stable.

Source note: The assessment states that the expected effort is approximately 4–5 hours and that incomplete work is acceptable when priorities are clearly documented. It also requires source code, frontend, README, database schema/migrations, API documentation or Postman collection, algorithm/state-transition explanations, demonstration instructions, tests, the exact “Assumptions / Questions / Requirement Issues” section, architectural decisions, and an “AI / Tool Usage” section.